

The 8.5° hover configuration: how the code produced it

The configuration that flies in every mode tested (headless SITL, live SITL, HITL on the Pixhawk 6X), the chain of scripts and package functions that produced it, and how the simulator represents it: the world and body frames, the attitude quaternion, and one simulation step. Each section points to the theory notebook that shows it with runnable code.

The configuration: `airframes/atlas_v3_30_jets_57.5_85_85_park-10.json` . V3_30 CAD masses (14.40 kg, CG at -0.114 / 0.003 / 0.089 m in the structural frame), foil jets leaving **57.5° / 85° / 85°** above the fan axis (inner / middle / outer station), parked at **-10°** on legs of 972 mm (rear) and 198 mm (front strut), hovering at **8.5°** nose-up. Rotation from the ground to hover 18.5°, tip-back angle 51.3°, busiest fan 68 % at hover, hover throttle 0.592.

These are *jet exit angles*, the target for the CFD and CAD work. No foil produces them yet: the CFD of the 75 / 65 / 50 foils measured a 39.6° jet.

What it does in flight

TEST	RESULT
Headless SITL, Stabilized, 3 seeds	3 / 3, drift 2.7 m (worst 3.3) in 60 s, touchdown 0.16 m/s
Live SITL, Altitude → Position → Altitude (after the clock fix)	Altitude drift 0.3-0.7 m, Position 0.02-0.07 m, touchdown 0.24 m/s
HITL on the 6X, Position on H-FLOW	held within 0.42 m for 60 s, touchdown 0.39 m/s
Static margins at hover	spare roll 12.9, pitch 22.8, yaw 1.35 N m; fans never saturate

1. The code workflow that made it



Figure 1. From the CAD file to a flying configuration. Each box is a script or a saved airframe; the airframe JSON is the single source every later step reads.

1 CAD to airframe

- `scripts/atlas_v3_30_from_step.py` . Reads `SMALL_SCALE_V3_30.step` through OpenCascade (`geometry/cad.py`), takes the masses from the top-level folder labels (`_M<g>g`), spreads each over its bodies by volume, sums the CG and inertia (parallel-axis), places the nine fans and the legs. Output: `airframes/atlas_v3_30.json` .
-

2 Gains

- `results/v3_30/fly.py` flies PX4 parameter variants headless in parallel; pitch-rate P 0.9 / I 0.05 won. Output: `atlas_v3_30_tuned.json` .
-

3 CFD thrust

- `scripts/atlas_v3_30_cfd.py` . Puts the OpenFOAM force (per foil group (-56.12, 46.48) N at 3×36 N in) on the foil fans: direction (cos e, 0, -sin e) with $e = 39.6^\circ$, magnitude through `turn_loss` . Output: `atlas_v3_30_cfd.json` .
-

4 Jet model

- `scripts/v3_30_foil_headroom.py` (`calibrate` , `jet` , `apply`). Fits two numbers so the CFD foils reproduce the CFD force: jet angle = $k \times$ foil angle ($k = 0.634$), thrust kept = $1 - c \cdot \text{loss}(\text{angle})$. Its `hover()` trims every candidate and scores it.
-

5 Jet sweep at park -10

- `scripts/v3_30_jet_sweep.py --park -10` . 9,261 inner / middle / outer jet triples ($40\text{--}90^\circ$, 2.5° steps) at two thrust-loss levels. Kept: hover $\leq 10^\circ$ (the 20° rotation limit from a -10° park), spare roll and pitch ≥ 5 N m, yaw ≥ 1 N m, still valid with any station 2.5° off. Picked: 57.5 / 85 / 85.
-

6 Build

- `v3_30_jet_sweep.build()` : `apply_jets` sets each fan's axis and loss; `Airframe.trim_hover_pitch()` finds 8.5° ; `Airframe.with_attitude(park_pitch_deg=-10)` re-solves the leg lengths (`geometry/gear.py: legs_for_park_pitch`) and checks the CG stays inside the feet; `hover_thrust_fraction()` gives MPC_THR_HOVER 0.592.
-

7 PX4 export

- `Airframe.px4_params()` : rotor positions and axes in the hover frame (`rotors_in_px4_frame`), relative thrust coefficients `CA_ROT0R*CT` , `SENS_BOARD_Y_OFF` = 8.5, nose-lift geometry, H-FLOW estimator offsets; `px4_params_sitl()` adds the SITL-only items.
-

8 Fly

- headless: `batch/worker.run_once` boots a private PX4 from a seeded working directory and drives `Simulator` + `ScenarioRunner` . Live: `server/app.py _scenario_start` (fresh PX4 with the clock restarted at 0, estimator check, scenario hooks). HITL: the same push to the board over USB.

How the hover pitch is found (step 6)

PX4 mixes with a pseudo-inverse. For a candidate hover pitch ϕ the code builds PX4's 6×9 effectiveness matrix B (rows: roll, pitch, yaw torque, then x, y, z force, all in the hover frame; one column per fan, `Airframe.effectiveness()`) and asks for pure upward thrust:

```

u = pinv(B) · [0, 0, 0, 0, 0, -1]          (unit vertical force, no torque)
util_i = u_i · (W / up) / Tmax_i           (scaled to carry the weight W =
141.2 N)
score(φ) = min( min util, 1 - max util )   (distance of the worst fan from 0
% or 100 %)

```

`trim_hover_pitch` scans ϕ from -10 to 80° and keeps the best score, rejecting pitches where the solution leaves a residual side force or torque. For this airframe the optimum is 8.5°, with every fan between 55 % and 68 %. Notebook: [06_fans_jetfoils_coanda §5](#).

2. World and body frames

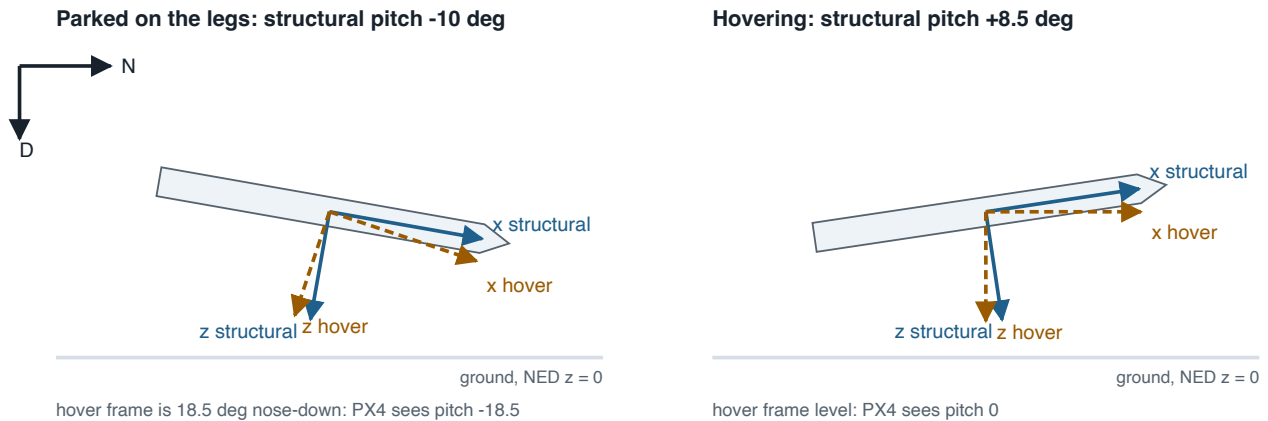


Figure 2. Side view. Solid blue: the structural frame fixed to the CAD. Dashed amber: the hover frame, what PX4 calls "body": the structural axes tilted 8.5° nose-down, so they are level exactly when the airframe hovers 8.5° nose-up. NED is fixed to the ground.

FRAME	DEFINITION IN THIS CODE	WHERE
World (NED)	Flat, non-rotating ground frame: x North, y East, z Down, origin on the ground under the start point (z = 0 is the floor). Treated as inertial: Earth rotation (7.3e-5 rad/s) and curvature are ignored at these speeds and ranges. The home latitude / longitude (Copenhagen by default) is used only to turn NED into GPS coordinates and to set the magnetic field.	<code>RigidBody.pos,</code> <code>vel (CG, NED);</code> <code>sensors/models.p</code> <code>y: Home</code>
Structural (body FRD)	Fixed to the airframe and its CAD: x Forward along the fuselage datum, y Right, z Down. Everything geometric is stored here: rotor positions and thrust axes, leg attach points, CG, inertia, the H-FLOW mount. The rigid body's attitude, rates and accelerometer reading are in this frame.	<code>Airframe.rotors[</code> <code>*.pos/axis,</code> <code>RigidBody.q,</code> <code>rates,</code> <code>accel_body</code>
Hover (PX4 body)	The structural frame rotated nose-up by the hover pitch $\phi = 8.5^\circ$ about y. When the aircraft hovers, this frame is level, so PX4's "level" is the hover attitude. The simulator still sends structural-frame IMU data; PX4 rotates it with <code>SENS_BOARD_Y_OFF = 8.5</code> .	<code>Airframe.hover_r</code> <code>otation(),</code> <code>RigidBody.hover_</code> <code>frame_euler()</code>
Rotor	Per fan: position and unit thrust axis in the structural frame. A jetfoil fan keeps its physical duct axis (1, 0, 0) and its push direction is the turned jet ($\cos e, 0, -\sin e$).	<code>geometry/propuls</code> <code>ion.py: Rotor</code>
Sensor	The H-FLOW: fixed 20.4° to the frame, so at hover it looks 11.9° off vertical; the rangefinder tilt goes to PX4 as <code>EKF2_RNG_PITCH = -0.2077</code> rad.	<code>sensors/flow.py:</code> <code>sensor_axes</code>

The rotation from structural to hover axes, for $\phi = 8.5^\circ$:

```

R_h = [[ cosφ, 0, sinφ],      = [[ 0.989, 0, 0.148],
      [ 0, 1, 0 ],          [ 0, 1, 0 ],
      [-sinφ, 0, cosφ]]      [-0.148, 0, 0.989]]
v_structural = R_h · v_hover

```

Worked through this configuration (structural → hover thrust axis): the outer and middle foil fans at 85° push along (0.087, 0, -0.996), which in the hover frame is (-0.061, 0, -0.998), almost straight up. The inner foil fans at 57.5° push (0.537, 0, -0.843) → (0.407, 0, -0.914), up and forward. The front centre fan M9 points straight down the structural z axis, (0, 0, -1) → (-0.148, 0, -0.989), up and slightly back. In hover the inner foils' forward push balances the backward lean of the nose fans and the outer foils; these hover-frame vectors are exactly what goes to PX4 as `CA_ROT0R*_AX/AY/AZ`. Notebook: [01_frames §1-4](#) (and §5 for the flow-gyro frame mismatch found in HITL).

3. Quaternions

The attitude is one unit quaternion, scalar first, $\mathbf{q} = [\mathbf{w}, \mathbf{x}, \mathbf{y}, \mathbf{z}]$, that rotates **structural (body) vectors into NED** (`dynamics/quaternion.py`). Its rotation matrix:

```

v_ned = R(q) · v_body
R(q) = [[1-2(y²+z²), 2(xy-wz), 2(xz+wy) ],
        [2(xy+wz), 1-2(x²+z²), 2(yz-wx) ],
        [2(xz-wy), 2(yz+wx), 1-2(x²+y²)]]

```

- **Hamilton convention**, the product $\mathbf{q} \otimes \mathbf{p}$ composes rotations; PX4 uses the same convention, so no sign flips at the interface.
- **Built from Euler angles** yaw-pitch-roll (ZYX): `q_from_euler(roll, pitch, yaw)`. At reset the vehicle is placed at the parked pitch: `q_from_euler(0, -10°, yaw) = [0.9962, 0, -0.0872, 0]`. Hovering level in the hover frame means structural pitch +8.5°: $\mathbf{q} = [0.9973, 0, 0.0741, 0]$.
- **How it moves** (`q_deriv`): the body rates $\omega = (p, q, r)$ drive it through the kinematic equation, integrated each step and renormalised:

$$d\mathbf{q}/dt = \frac{1}{2} \mathbf{q} \otimes [0, p, q, r] \quad \mathbf{q} \leftarrow \text{normalize}(\mathbf{q} + d\mathbf{q}/dt \cdot dt)$$

- **Why a quaternion** and not Euler angles: Euler angles hit a singularity at 90° pitch (gimbal lock), and this aircraft parks and hovers at large pitches in other variants (43-50°). The quaternion has no singular attitude; Euler angles are only computed for display and logging (`q_to_euler`).
- **What PX4 sees** (`hover_frame_euler`): $\mathbf{R}_p = \mathbf{R}(\mathbf{q}) \cdot \mathbf{R}_h^T$ is the attitude of the hover frame; converted back to a quaternion and then Euler angles it gives the roll / pitch PX4 reports. Parked: PX4 pitch -18.5°; hovering: 0°.

Notebook: [02_quaternions §1-5](#).

4. One simulation step

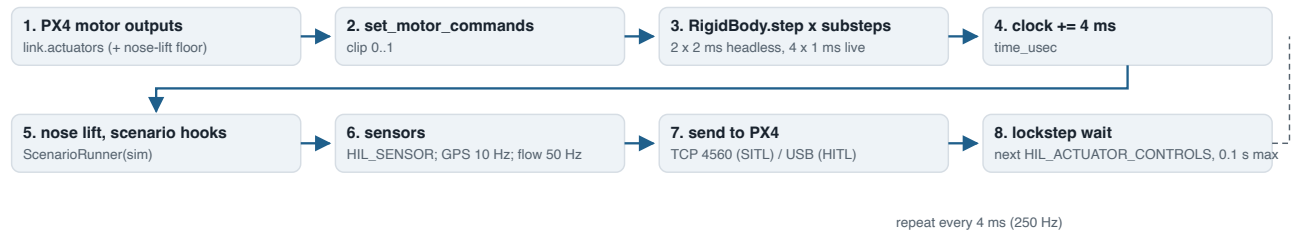


Figure 3. One sensor step of `Simulator.step_once()` (`sim/simulator.py`), repeated at 250 Hz. In SITL PX4 runs in lockstep: its clock only advances with each sensor message.

The outer loop (sensor step, 4 ms). Read PX4's last motor outputs (the nose lift adds a floor under the nose fans while it rotates the aircraft), run the physics in sub-steps, advance the clock, let the nose-lift executor and the scenario runner act (sticks, modes, phase changes, metrics), build the sensor messages from the true state, send them, then wait for PX4's next motor outputs. Headless runs go as fast as PX4 answers; the live app paces to real time. Since the drift fix the clock restarts at zero before every SITL PX4 launch (`Simulator.restart_clock`).

The physics (`RigidBody.step`, `dynamics/rigid_body.py`), 2 ms headless or 1 ms live:

#	COMPUTATION	MEANING
1	<code>F_body, M_body</code> from fans (thrust along each axis, ram drag), wings, body drag <code>F_ned = R(q) F_body + [0, 0, m g] + legs</code>	forces in the structural frame about the CG, moved to NED; gravity and the leg springs, dampers and friction added there
2	<code>a = F_ned / m</code> <code>α = I⁻¹ (M_body - ω × Iω)</code>	Newton ($m = 14.40$ kg); Euler's rotation equation, I constant in the body frame
3	<code>Ω ← Ω + (cmd - Ω) / τ · dt</code>	fan speed follows the command, first order, $\tau = 0.12$ s
4	<code>v ← v + a dt</code> , then <code>p ← p + v dt</code> <code>ω ← ω + α dt</code> , then <code>q ← normalize(q + ½ q ⊗ [0, ω] dt)</code>	semi-implicit Euler: the new velocity moves the position, the new rates turn the attitude
5	<code>accel = R(q)^T (Δv/dt - [0, 0, g])</code>	the accelerometer's specific force, from the motion actually made this step

- **Why this split:** translation lives in NED, where gravity is a constant vector and the ground is $z = 0$; rotation lives in the body frame, where the inertia tensor ($0.87 / 1.11 / 1.73$ kg m², plus products) does not change as the aircraft turns.
- **Semi-implicit Euler** (velocity first, then position with the new velocity) is stable for the stiff leg springs at 1-2 ms, where plain Euler gains energy.
- **Legs** are algebraic penalty springs, not constraints: the force comes from how far each foot is below the floor and how fast it moves, so contact needs no extra state (`dynamics/contact.py`).
- **Sensors** (`sensors/models.py: hil_sensor`): accelerometer and gyro (with noise and the fan-vibration model), magnetometer from the dipole field rotated into the body, barometer from the ISA atmosphere; GPS at 10 Hz; the H-FLOW flow and range at 50 Hz (`sensors/flow.py`).

Notebooks: [03_equations_of_motion §1, 2, 5, 6](#) (state, equations, spool lag, integrator), [04_ground_contact](#) (legs, parked angle), [05_sensors](#) (accelerometer, HIL_SENSOR, H-FLOW).

Notebook guide

NOTEBOOK	READ IT FOR
00_index	conventions and how to run them
01_frames	NED, structural, hover, rotor and sensor frames; the 8.5° rotation; the flow-gyro mismatch
02_quaternions	body-to-NED quaternion, Hamilton product, kinematics, gimbal lock, hover-frame angles
03_equations_of_motion	state vector, Newton-Euler, inertia and parallel-axis, spool lag, the integrator
04_ground_contact	leg springs, damping clamp, setting the parked angle (the -10° park)
05_sensors	what PX4 receives: specific force, HIL_SENSOR, H-FLOW
06_fans_jetfoils_coanda	turned jets, loss, Coanda, ram drag, the mixer export and hover trim
07_jsbsim_and_chrono	the alternative physics engines

Open them with the kernel "AIRFRAME_DESIGNER (.venv)". They load `airframes/atlas_v3_v34_foils_50_65_50_tuned.json` in their setup cell; to see every number for this configuration, set `AF_PATH` in `notebooks/_build_notebooks.py` to `airframes/atlas_v3_30_jets_57.5_85_85_park-10.json` and rebuild (the source of truth is that script, not the .ipynb files). Long-form theory: `docs/ATLAS_Simulator_Physics.pdf`.

Files

Configuration `airframes/atlas_v3_30_jets_57.5_85_85_park-10.json`; sweep `results/v3_30/jet_sweep/20261001_173459/` (`jet_static.json` , `summary.json` , flights); scenarios `2026-10-01_1627..._fastlift` (Stabilized), `2026-10-01_1851..._poshold_hflow` (Position), `2026-10-01_1926..._altitude_position_altitude`; live flights `results/tuning/live9097*.json` , `live9098*.json`; HITL `live907599734`. Related reports: `V3_30_Foil_Angles_20261001.pdf` (the jet targets), `V3_30_Live_Drift_Root_Cause_20261001.pdf` (the clock fix).

ATLAS V3_30 simulator, PX4 1.17, 250 Hz sensors. Generated 2026-10-01.